INSTITUT UNIVERSITAIRE DE TECHNOLOGIE VALENCIENNES • CAMBRAI • MAUBEUGE

Université Polytechnique HAUTS-DE-FRANCE

SAE S1.01 — Jeu MasterMind

Rédigé par:
Brian.loubassou

Année universitaire 2026/2027

Plan de presentation

1. Présentation du projet

   - 1.1 Présentation et contexte du projet
   - 1.2 Objectifs du projet
   - 1.3 Outils utilisés

2. Présentation du fonctionnement du jeu

   - 2.1 Règles du MasterMind
   - 2.2 Fonctionnement du programme
   - 2.3 Interface graphique

3. Présentation des fonctions

   - 3.1 Fonction `distance`
     - 3.1.1 Rôle de la fonction
     - 3.1.2 Algorithme en pseudo-code
     - 3.1.3 Tableau des variables
     - 3.1.4 Tests de la fonction
   - 3.2 Fonction `genererCombinaison`
     - 3.2.1 Rôle de la fonction
     - 3.2.2 Algorithme en pseudo-code
     - 3.2.3 Tableau des variables
   - 3.3 Fonction `calcul résultat`
     - 3.3.1 Rôle de la fonction
     - 3.3.2 Algorithme en pseudo-code
     - 3.3.3 Tableau des variables
     - 3.3.4 Tests de la fonction

4. Tests et vérification du programme

   - 4.1 Tests du fonctionnement général
   - 4.2 Résultats obtenus

5. Conclusion

1. Présentation du projet

1.1 Présentation et contexte du projet

Dans le cadre de la SAE S1.01, nous devons réaliser un programme en Python permettant de jouer une partie de MasterMind contre l'ordinateur.

Le MasterMind est un jeu de réflexion dans lequel un joueur doit retrouver une combinaison secrète de couleurs. Dans notre projet, c'est l'ordinateur qui génère cette combinaison, composée de cinq couleurs, et le joueur doit essayer de la deviner.

Pour réaliser ce projet, nous utilisons le langage Python ainsi que la bibliothèque Pygame, qui permet de créer une interface graphique. Nous utilisons également le fichier mm.py, fourni dans le sujet, qui contient plusieurs fonctions permettant d'afficher le plateau de jeu, de choisir les couleurs et d'afficher les résultats.

Ce projet nous permet de mettre en pratique les notions de programmation étudiées en cours, notamment les fonctions, les listes, les boucles, les conditions et les tests. Il nous permet également d'apprendre à organiser un programme en plusieurs parties et à vérifier son bon fonctionnement.

1.2 Objectifs du projet

L'objectif principal de ce projet est de créer un jeu MasterMind en Python dans lequel le joueur doit deviner une combinaison secrète générée aléatoirement par l'ordinateur.

Pour réaliser cet objectif, plusieurs étapes sont nécessaires :

- Générer une combinaison secrète composée de cinq couleurs.
- Afficher le plateau de jeu et les couleurs disponibles grâce à la bibliothèque mm.py.
- Permettre au joueur de proposer une combinaison de cinq couleurs.
- Comparer la proposition du joueur avec la combinaison secrète.
- Calculer le nombre de pions bien placés et le nombre de pions mal placés.
- Afficher le résultat après chaque proposition.
- Indiquer si le joueur a gagné ou perdu et, en cas de victoire, afficher le nombre de coups utilisés.
- Attendre que le joueur ferme le programme.

Ce projet a également pour objectif de mettre en pratique les notions de programmation étudiées en cours, comme les fonctions, les listes, les boucles, les conditions et les tests. Il permet ainsi de développer un programme organisé et de vérifier son bon fonctionnement.

1.3 Outils utilisés

Pour réaliser ce projet, nous avons utilisé plusieurs outils et technologies.

Python : c'est le langage de programmation utilisé pour développer le jeu. Il permet de créer les fonctions, de gérer les combinaisons de couleurs et de vérifier les propositions du joueur.

Pygame : c'est une bibliothèque Python utilisée pour créer l'interface graphique du jeu. Elle permet d'afficher la fenêtre, le plateau, les couleurs et les résultats des propositions.

Le module mm.py : ce fichier est fourni dans le sujet de la SAE. Il contient des fonctions permettant de gérer l'affichage du plateau, le choix des couleurs, la construction des propositions et l'affichage des résultats. Nous utilisons ces fonctions dans notre programme principal grâce à l'instruction import mm.py

Un environnement de développement : nous utilisons un éditeur de code Python pour écrire, modifier et exécuter le programme, ainsi que pour tester son fonctionnement.

2. Présentation du fonctionnement du jeu

2.1 Règles du MasterMind

Le MasterMind est un jeu de réflexion dans lequel le joueur doit retrouver une combinaison secrète de couleurs.

Dans notre projet, l'ordinateur génère aléatoirement une combinaison de cinq couleurs parmi les huit couleurs disponibles. Le joueur doit retrouver cette combinaison en proposant différentes suites de cinq couleurs.

Après chaque proposition, le programme indique le nombre de pions bien placés et le nombre de pions mal placés. Un pion bien placé correspond à une couleur correcte située au bon endroit. Un pion mal placé correspond à une couleur présente dans la combinaison secrète, mais située à un autre endroit.

Le joueur dispose de quinze essais pour trouver la combinaison secrète. Il gagne s'il trouve les cinq couleurs dans le bon ordre. Sinon, il perd lorsque les quinze essais sont terminés.

2.2 Fonctionnement du programme

Le programme fonctionne en plusieurs étapes :

1. Il initialise Pygame et crée la fenêtre du jeu.
2. Il affiche le plateau de jeu et les huit couleurs disponibles.
3. Il génère aléatoirement une combinaison secrète de cinq couleurs.
4. Le joueur choisit cinq couleurs pour former une proposition.
5. Le programme compare cette proposition à la combinaison secrète.
6. Il calcule le nombre de pions bien placés et mal placés.
7. Il affiche le résultat sur le plateau.
8. Il vérifie si le joueur a gagné ou s'il a atteint le nombre maximal d'essais.
9. À la fin de la partie, il affiche un message de victoire ou de défaite, puis attend que le joueur ferme le programme.

Ces étapes se répètent tant que la partie n'est pas terminée.

2.3 Interface graphique

L'interface graphique est réalisée avec la bibliothèque Pygame et les fonctions fournies dans le fichier mm.py.

La fenêtre contient plusieurs éléments :

- **Le plateau de jeu :** il permet d'afficher les propositions successives du joueur.
- **Les couleurs disponibles :** elles sont affichées sur le côté gauche de la fenêtre et permettent au joueur de choisir les couleurs de sa proposition.
- **La combinaison secrète :** elle est affichée en haut du plateau à la fin de la partie.
- **Les résultats :** des petits pions blancs indiquent le nombre de couleurs bien placées, tandis que des petits pions noirs indiquent le nombre de couleurs mal placées.
- **Les messages :** ils indiquent au joueur s'il a gagné ou perdu.

Le joueur utilise sa souris pour sélectionner les couleurs et peut retirer le dernier pion choisi grâce au bouton prévu à cet effet. Une fois les cinq couleurs sélectionnées, le programme calcule et affiche le résultat de la proposition.

3. Présentation des fonctions

Dans cette partie, nous allons présenter les fonctions utilisées dans notre programme MasterMind. Chaque fonction permet de réaliser une tâche précise afin de faciliter le fonctionnement du jeu et de rendre le code plus organisé.

La fonction `genererCombinaison()` permet de créer aléatoirement la combinaison secrète de cinq couleurs. La fonction `calculresultat()` compare la proposition du joueur avec la combinaison secrète afin de déterminer le nombre de pions bien placés et mal placés. La fonction `afficherMessage()` permet d'afficher un message à l'écran, notamment lorsque le joueur gagne ou perd.

Nous présenterons également la fonction `distance()`, située dans le fichier `mm.py`, qui permet de calculer la distance entre deux points. Elle est utilisée pour détecter les clics de la souris sur les couleurs de l'interface graphique.

3.1 Fonction distance

```
def distance(a:list,b:list)->float:
    return math.sqrt((a[0] - b[0])**2 + (a[1] - b[1])**2)
```

3.1.1 Rôle de la fonction

La fonction distance(a, b) permet de calculer la distance entre deux points à partir de leurs coordonnées. Chaque point est représenté par une liste contenant deux valeurs : l'abscisse et l'ordonnée.

Cette fonction est utilisée dans le module mm.py pour vérifier si la souris se trouve suffisamment près d'une couleur lorsque le joueur clique dessus. Si la distance

entre la souris et le centre d'un pion est inférieure à 15 pixels, le clic est considéré comme valide.

La fonction utilise la formule mathématique de la distance entre deux points et retourne le résultat sous forme d'un nombre décimal.

3.1.2 Algorithme en pseudo-code

Fonction distance(a, b)

    calculer la différence entre les abscisses

    calculer la différence entre les ordonnées

    élever les deux différences au carré

    additionner les deux carrés

    calculer la racine carrée du résultat

    retourner la distance

Fin Fonction

3.1.3 Tableau des variables

Variable	Type	Rôle
a	Liste de deux nombres	Coordonnées du premier point
b	Liste de deux nombres	Coordonnées du deuxième point

`a[0]` et `b[0]`	Nombre	Abscisses des deux points
`a[1]` et `b[1]`	Nombre	Ordonnées des deux points
Valeur retournée	Nombre décimal (`float`)	Distance entre les deux points

3.1.4 Tests de la fonction

```
import mm

print("Test 1 :", mm.distance([0, 0], [0, 0]))
print("Test 2 :", mm.distance([0, 0], [3, 4]))
print("Test 3 :", mm.distance([1, 2], [4, 6]))
print("Test 4 :", mm.distance([0, 0], [0, 5]))
```

Points testés	**Résultat attendu**	**Résultat obtenu**
`[0, 0]` et `[0, 0]`	0	0.0
`[0, 0]` et `[3, 4]`	5	5.0
`[1, 2]` et `[4, 6]`	5	5.0
`[0, 0]` et `[0, 5]`	5	5.0

Résultat obtenu :

```
pygame 2.6.1 (SDL 2.28.4, Python 3.10.11)
Hello from the pygame community. https://www.pygame.org/contribute.html
Test 1 : 0.0
Test 2 : 5.0
Test 3 : 5.0
Test 4 : 5.0
>>>
```

3.2. La fonction `genererCombinaison()`

```python
def genererCombinaison():
    """
    Cette fonction permet de générer la combinaison secrète.
    """
    secret = []

    for i in range(5):
        secret.append(mm.TabCouleur[random.randint(0, 7)])

    return secret

test = genererCombinaison()
print(test)
print(len(test))
```

3.2.1 Rôle de la fonction

La fonction genererCombinaison() permet de générer la combinaison secrète que le joueur doit deviner. Cette combinaison contient cinq couleurs choisies aléatoirement parmi les huit couleurs disponibles dans le module mm.py.

Pour cela, le programme crée une liste vide appelée secret, puis ajoute cinq couleurs sélectionnées au hasard grâce à la fonction random.randint(0, 7). La liste des couleurs disponibles est mm.TabCouleur.

Cette fonction est nécessaire pour que chaque partie puisse commencer avec une nouvelle combinaison secrète.

3.2.2 Algorithme en pseudo-code

Début

Créer une liste vide appelée secret

Répéter 5 fois :

Choisir un nombre aléatoire entre 0 et 7

Récupérer la couleur correspondante dans mm.TabCouleur

Ajouter cette couleur à secret

Retourner secret

Fin

3.2.3 Tableau des variables

Nom de la Variable	Type	Role
Secret	Liste	Contient les cinq couleurs de la combinaison
i	Entier	Compteur permettant de répéter l'opération cinq fois

La fonction utilise également `mm.TabCouleur`, qui est une liste contenant les huit couleurs disponibles, et `random.randint(0, 7)`, qui permet de choisir un indice aléatoire dans cette liste.

Vérification de la fonction

Pour vérifier que la fonction fonctionne correctement, on peut l'exécuter plusieurs fois et observer les combinaisons obtenues.

Test	Résultat attendu

Appeler genererCombinaison()	La fonction retourne une liste de cinq couleurs
Vérifier le nombre de couleur	La liste contient exactement cinq éléments
Vérifier les couleurs obtenue	Chaque couleur appartient à mm.TabCouleur
Appeler la fonction plusieurs fois	Les combinaison peut etre differentes d'un appel à l'autre

Ces tests permettent de vérifier que la combinaison contient bien cinq couleurs autorisées. Les combinaisons peuvent comporter plusieurs fois la même couleur, car le programme effectue cinq tirages indépendants.

3.3. La fonction calculresultat()

```
24
25 def calculresultat(proposition, secret):
26     """
27     Cette fonction calcule le nombre de pions bien placés
28     et le nombre de pions mal placés.
29     """
30
31     bien = 0
32     pasbien = 0
33
34     combinaison_temp = list(secret)
35
36     # recherche des pions bien placés
37     for i in range(5):
38         if proposition[i] == secret[i]:
39             bien = bien + 1
40             combinaison_temp[i] = None
41
42     # recherche des pions mal placés
43     for elt in proposition:
44         if elt in combinaison_temp:
45             pasbien = pasbien + 1
46             position = combinaison_temp.index(elt)
47             combinaison_temp[position] = None
48
49     return bien, pasbien
50
```

3.3.1 Rôle de la fonction

La fonction calculresultat(proposition, secret) compare la combinaison proposée par le joueur avec la combinaison secrète. Elle permet de compter les pions bien placés et les pions mal placés.

Un pion est bien placé si sa couleur correspond à celle de la combinaison secrète et qu'il se trouve à la même position. Un pion est mal placé si sa couleur est présente dans la combinaison secrète, mais qu'il se trouve à une autre position.

Au début, les variables bien et pasbien sont initialisées à zéro. La fonction crée ensuite une copie de la combinaison secrète, appelée combinaison_temp.

Dans un premier temps, elle recherche les pions bien placés. Lorsqu'un pion est bien placé, elle augmente bien de 1 et remplace la couleur correspondante par None dans combinaison_temp. Cela évite de compter ce pion une deuxième fois.

Dans un deuxième temps, elle recherche les pions mal placés. Pour chaque couleur de la proposition, elle vérifie si cette couleur est présente dans combinaison_temp. Si c'est le cas, elle augmente pasbien de 1 et remplace la couleur trouvée par None pour éviter de la compter plusieurs fois.

Enfin, la fonction retourne un tuple contenant les deux résultats : (bien, pasbien).

3.3.2 Algorithme en pseudo-code

Début

bien ← 0

pabien ← 0

combinaison_temp ← copie de secret

Pour i allant de 0 à 4 :

Si proposition[i] = secret[i] :

bien ← bien + 1

combinaison_temp[i] ← None

Pour chaque elt dans proposition :

Si elt est dans combinaison_temp :

pabien ← pabien + 1

position ← position de elt dans combinaison_temp

combinaison_temp[position] ← None

Retourner (bien, pabien)

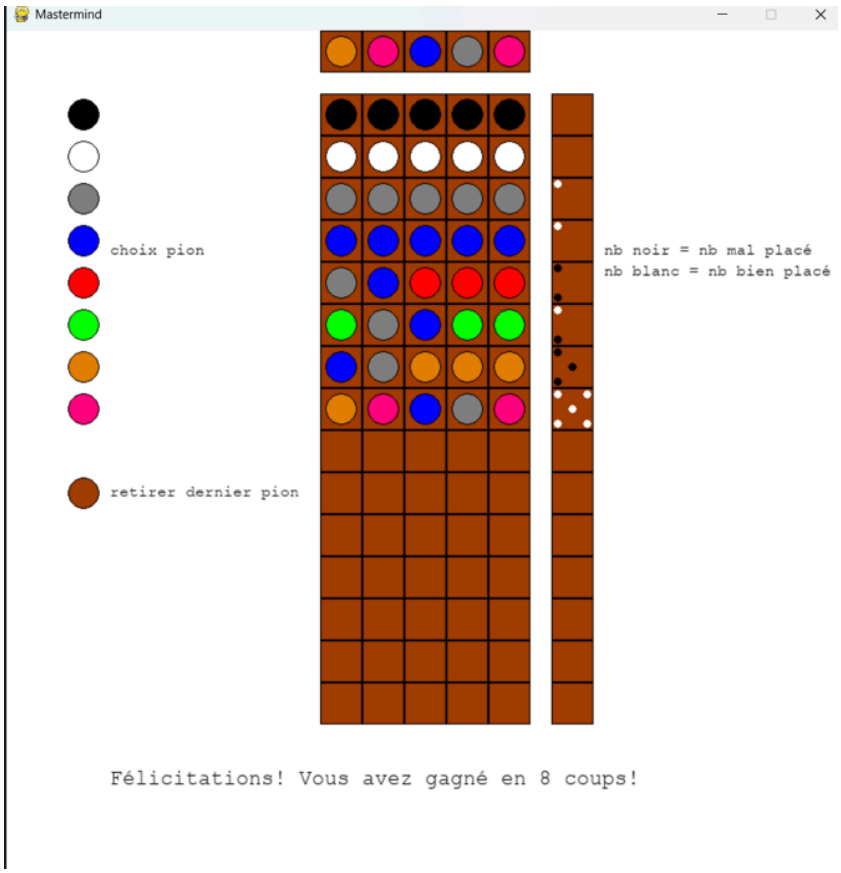
Fin

3.3.3 Tableau des variables

Nom	Type	Role
proposition	Liste	Continent les cinq couleurs choisie par le joueurs
Secret	Liste	Contient la combinaison secret à deviner
Bien	Entier	Compte les pion bien placés
pasbien	Entier	Compte les pion mal placés
Combinaison_temps	Liste	Copie de la combinaison secrète utilisée pour éviter de compter plusieurs fois un même pion

i	Entier	Indice utilisé pour parcourir les cinq position
els	Tupe d'entiers	Représente une couleur parcourue dans la proposition,car les couleurs sont codées sous forme de tuples RGB
Position	Entier	Indique l'indice de la couleur trouvée dans combinaison_temp

Tests de la fonction



Les tests suivants permettent de vérifier le calcul des pions bien placés et mal placés. Pour faciliter la lecture, les couleurs sont désignées par leur nom.

Test	**Combinaison secret**	**Proposition du joueur**	**Resultat attendu**
1 – Couleurs absentes	Orange,rose,bleu,gris,rose	Noir,noir,noir,noir,noir	(0,0)
2 – Couleurs absentes	Orange,rose,bleu,gris,rose	Blanc,blanc,blanc,blanc,blanc	(0,0)
3 – Couleur répétée	Orange,rose,bleu,gris,rose	Gris, gris, gris, gris, gris	(0,1)
4 – Couleur répétée	Orange,rose,bleu,gris,rose	Bleu, bleu, bleu, bleu, bleu	(0,1)
5 – Plusieurs couleurs déplacées	Orange,rose,bleu,gris,rose	Gris, bleu, rouge, rouge, rouge	(0,2)
6 – Couleurs répétées	Orange,rose,bleu,gris,rose	Vert, gris, bleu, vert, vert	(0,2)
7 – Plusieurs couleurs déplacées	Orange,rose,bleu,gris,rose	Bleu, gris, orange, orange, orange	(1,2)
8 – Combinaison identique	Orange,rose,bleu,gris,rose	Orange,rose,bleu,gris,rose	(5,0)

Le test 1 vérifie que les couleurs noires ne sont pas comptées lorsqu'elles sont absentes de la combinaison secrète. Le test 2 vérifie le même cas avec les pions blancs. Les tests 3 et 4 permettent de vérifier que le programme ne compte pas plusieurs fois une couleur répétée. Les tests 5, 6 et 7 vérifient le comptage des couleurs mal placées, même lorsque certaines couleurs sont répétées. Enfin, le test

8 vérifie que cinq pions bien placés donnent le résultat `(5, 0)`, ce qui correspond à une victoire.

3.4. La fonction `afficherMessage()`

```
def afficherMessage(f: pygame.Surface, message: str) -> None:
    myfont = pygame.font.SysFont("monospace", 20)
    label = myfont.render(message, 1, mm.Noir)
    f.blit(label, (100, 700))
    pygame.display.update()



#permet affichage du jeu grace a pygame
laFenetre = pygame.display.set_mode((800, 800))
pygame.display.set_caption("Mastermind")
laFenetre.fill(mm.Blanc)
mm.afficherPlateau(laFenetre)
mm.afficherChoixCouleur(laFenetre)



combinaison = combinaison()
#mm.afficherSecret(laFenetre,combinaison)
for ligne in range(2, 17):
    proposition = mm.construireProposition(laFenetre, ligne)
    resultat = calculresultat(proposition, combinaison)
    mm.afficherResultat(laFenetre, resultat, ligne)
    if proposition == combinaison:
        break
    if ligne == 16 and combinaison != proposition:
        message = "Désolé, vous avez perdu. Appuyer sur entrer pour fermer"
        afficherMessage(laFenetre, message)
```

```
mm.afficherSecret(laFenetre,combinaison)



#pour quiter la page
fini = False
while not fini:
    for event in pygame.event.get():
        if event.type == pygame.KEYDOWN:
            keys = pygame.key.get_pressed()
            fini = keys[pygame.K_RETURN]
            print("appuie sur une touche")
        if event.type == pygame.QUIT:
            fini = True

pygame.quit()
```

3.4.1 Rôle de la fonction

Rôle de la fonction

La fonction `afficherMessage(f, message)` permet d'afficher un texte dans la fenêtre du jeu. Dans notre programme, elle est utilisée pour informer le joueur qu'il a perdu et qu'il doit appuyer sur la touche Entrée pour fermer la fenêtre.

La fonction utilise Pygame pour créer une police de caractères de type `monospace`, de taille 20. Elle transforme ensuite le message en une image avec `myfont.render()`, en utilisant la couleur noire définie dans `mm.Noir`.

Le texte est affiché aux coordonnées `(100, 700)` grâce à `f.blit()`. Enfin, `pygame.display.update()` actualise la fenêtre afin que le message apparaisse à l'écran.

Cette fonction permet de prévenir le joueur de la fin de la partie sans devoir répéter les instructions d'affichage dans le programme principal.

3.4.2 Algorithme en pseudo-code

Début

Créer une police monospace de taille 20

Transformer le message en image noire

Afficher le message aux coordonnées (100, 700)

Actualiser la fenêtre

Fin

Tableau des variables

Nom	Type	Role
f	pygame.surface	Surface sur laquelle le message est affiché
message	Chaine de caracteres(str)	Texte à afficher à joueur
myfont	Objet police pygame	Définit la police et la taille du texte
label	Surface pygame	Contient le texte transformé en image

Patie 2 Le module mm.py

3.5. Les fonctions du module `mm.py`

Le module mm.py regroupe les fonctions qui permettent de créer l'interface graphique du MasterMind et de gérer les interactions avec le joueur. Il utilise la bibliothèque Pygame pour dessiner les pions, le plateau et les résultats des propositions.

3.5.1. Les variables globales

Au début du fichier, plusieurs variables sont définies pour représenter les couleurs utilisées dans le jeu. Chaque couleur est représentée par un tuple de trois nombres correspondant aux composantes rouge, vert et bleu (RGB).

```
# importation du module pygame
import pygame
import random
import math


#definition de variables globales
Noir = (0,0,0)
Blanc = (255,255,255)
Gris = (128,128,128)
Bleu = (0,0,255)
Rouge = (255,0,0)
Vert = (0,255,0)
Orange = (225,127,0)
Rose = (255,0,127)

Marron = (160,60,0)



TabCouleur  = [Noir,Blanc,Gris,Bleu,Rouge,Vert,Orange,Rose]
```

3.5.2. La fonction afficherSecret()

Cette fonction affiche la combinaison secrète sous forme de cinq pions en haut du plateau. Elle parcourt la liste `leSecret` et dessine chaque couleur avec `pygame.draw.circle()`. Un contour noir est ensuite ajouté à chaque pion pour le rendre plus visible.

Dans le déroulement du jeu, cette fonction permet de révéler la combinaison secrète à la fin de la partie.

```
def afficherSecret(laFenetre:pygame.Surface,leSecret:list)->None:
    for i in range(len(leSecret)):
        pygame.draw.circle(laFenetre,leSecret[i],[320+40*i,20],15)
        pygame.draw.circle(laFenetre,Noir,[320+40*i, 20],15,1)
    pygame.display.update()
```

3.5.3. La fonction `afficherCombinaison()`

Cette fonction affiche la proposition du joueur sur une ligne précise du plateau. Elle reçoit la fenêtre, la liste des couleurs choisies et le numéro de la ligne.

Elle dessine d'abord les emplacements, puis les pions correspondant aux couleurs sélectionnées. Chaque pion est entouré d'un contour noir. La fenêtre est ensuite actualisée pour afficher la proposition.

```
def afficherCombinaison(laFenetre:pygame.Surface,laCombinaison:list,numLigne:int)->None:

    for i in range(5):
        pygame.draw.circle(laFenetre,Marron,[320+40*i,40+40*(numLigne-1)],15)

    for i in range(len(laCombinaison)):
        pygame.draw.circle(laFenetre,laCombinaison[i],[320+40*i,40+40* (numLigne-1)],15)
        pygame.draw.circle(laFenetre,Noir,[320+40*i, 40+40* (numLigne-1)],15,1)

    pygame.display.update()
```

3.5.4. La fonction afficherPlateau()

Cette fonction dessine le plateau de jeu, les cases des quinze essais et la colonne réservée aux résultats. Elle affiche également des indications pour aider le joueur à comprendre le fonctionnement du jeu, notamment la signification des pions noirs et blancs.

Elle permet de préparer l'interface graphique avant le début de la partie.

```
def afficherPlateau(f:pygame.Surface)->None:
    pygame.draw.rect(f,Marron,[300,0,200,40])
    for i in range(5):
        pygame.draw.rect(f,Noir,[300+40*i,0,40,40],1)
    pygame.draw.rect(f,Marron,[300,60,200,40*15])
    pygame.draw.rect(f,Marron,[520,60,40,40*15])

    for l in range(15):
        for i in range(5):
            pygame.draw.rect(f,Noir,[300+40*i,60+40*l,40,40],1)
        pygame.draw.rect(f,Noir,[520,60+40*l,40,40],1)

    text1 = "nb noir = nb mal placé"
    text2 = "nb blanc = nb bien placé"
    text3 = "choix pion"
    text4 = "retirer dernier pion"
    myfont = pygame.font.SysFont("monospace", 15)
    label1 = myfont.render(text1, 1, Noir)
    label2 = myfont.render(text2, 1, Noir)
    label3 = myfont.render(text3, 1, Noir)
    label4 = myfont.render(text4, 1, Noir)
    f.blit(label3, (100, 200))
    f.blit(label4, (100, 430))
    f.blit(label1, (570, 200))
    f.blit(label2, (570, 220))
    pygame.display.update()
```

3.5.5. La fonction afficherChoixCouleur()

Cette fonction affiche les huit couleurs disponibles sur la gauche de la fenêtre. Elle dessine également un pion marron qui sert à retirer le dernier pion ajouté à la proposition.

Le joueur peut ainsi sélectionner les couleurs nécessaires pour construire sa combinaison.

```
67  def afficherChoixCouleur(f:pygame.Surface)->None:
68      for i in range(len(TabCouleur)):
69          pygame.draw.circle(f,TabCouleur[i],[75,80+40*i],15)
70          pygame.draw.circle(f,Noir,[75,80+40*i],15,1)
71      pygame.draw.circle(f,Marron,[75,80+40*9],15)
72      pygame.draw.circle(f,Noir,[75,80+40*9],15,1)
73      pygame.display.update()
74
```

3.5.6. La fonction getChoixCouleur()

```
78  def getChoixCouleur()->None:
79      while True:
80          for event in pygame.event.get():
81              if event.type == pygame.QUIT:
82                  print("c'est fini!!!")
83                  pygame.quit()
84              if event.type == pygame.MOUSEBUTTONUP:
85                  positionSouris = pygame.mouse.get_pos()
86                  if distance(positionSouris, [75,80+40*9])<15:
87                      return None
88                  for i in range(len(TabCouleur)) :
89                      if distance(positionSouris, [75,80+40*i])<15:
90                          return TabCouleur[i]
91
```

Cette fonction attend que le joueur clique sur une couleur avec la souris. Elle récupère la position du curseur et utilise la fonction distance() pour vérifier si le clic se trouve suffisamment près du centre d'un pion.

Si une couleur est sélectionnée, la fonction la retourne. Si le joueur clique sur le pion marron, elle retourne None, ce qui permet de demander la suppression du dernier pion de la proposition.

3.5.7. La fonction `construireProposition()`

Cette fonction permet au joueur de construire sa proposition en sélectionnant cinq couleurs. Elle appelle `getChoixCouleur()` pour récupérer chaque choix, puis ajoute les couleurs à la liste `proposition`.

Si le joueur sélectionne le pion marron, le dernier élément de la liste est supprimé, à condition que la liste ne soit pas vide. La proposition est ensuite affichée sur la ligne correspondante grâce à `afficherCombinaison()`.

Lorsque cinq couleurs ont été sélectionnées, la fonction retourne la proposition au programme principal.

```
92
93  def construireProposition(f:pygame.Surface,ligne:int)->list:
94      proposition = []
95
96      while len(proposition)<5:
97          for event in pygame.event.get():
98              if event.type == pygame.QUIT:
99                  pygame.quit()
100         couleur = getChoixCouleur()
101         if couleur==None:
102             if len(proposition)>0:
103                 del proposition[-1]
104         else:
105             proposition.append(couleur)
106         afficherCombinaison(f,proposition,ligne)
107         pygame.display.update()
108
109     return proposition
110
111
```

3.5.8. La fonction `afficherResultat()`

```
111
112
113 def afficherResultat(f:pygame.Surface,res,ligne):
114     x = 520
115     y = 20+40*(ligne-1)
116     centres = [(x+6,y+6),(x+6,y+34),(x+20,y+20),(x+34,y+6),(x+34,y+34)]
117     i = 0
118     while i<res[0]:
119         pygame.draw.circle(f,Blanc,centres[i],4)
120         i = i + 1
121     j = 0
122     while j<res[1]:
123         pygame.draw.circle(f,Noir,centres[i],4)
124         i = i + 1
125         j = j + 1
126     pygame.display.update()
127
128 pygame.init()
```

Cette fonction affiche le résultat obtenu après chaque proposition. Elle reçoit le résultat du calcul sous la forme d'un tuple contenant le nombre de pions bien placés et le nombre de pions mal placés.

Les pions blancs représentent les couleurs bien placées, tandis que les pions noirs représentent les couleurs mal placées. La fonction dessine les petits cercles correspondants dans la colonne des résultats, puis actualise la fenêtre.

Elle permet au joueur d'utiliser les indications obtenues pour améliorer ses propositions suivantes.

Remarque :

Dans afficherResultat(), on utilises res[0] pour les pions blancs et res[1] pour les pions noirs. Cela correspond bien à non calcul, où res[0] représente les pions bien placés et res[1] les pions mal placés.

Résumé général du projet

Ce projet consiste à programmer le jeu MasterMind en Python avec la bibliothèque Pygame. Le programme génère une combinaison secrète de cinq couleurs que le joueur doit retrouver en quinze essais maximum. À chaque tentative, le joueur choisit ses couleurs grâce à une interface graphique, puis le programme indique le nombre de pions bien placés et mal placés. Le module `mm.py` contient les fonctions qui permettent d'afficher le plateau, de gérer les choix du joueur et d'afficher les résultats. D'autres fonctions servent à générer la combinaison secrète et à calculer le résultat des propositions. Enfin, la fonction `distance()` permet de calculer la distance entre deux points afin de détecter les clics sur les couleurs. L'objectif du projet est donc de mettre en pratique les bases de Python, les fonctions, les listes et la création d'une interface graphique interactive.